

University
Politehnica
of Bucharest





Computer
Science and
Engineering
Department

Cassandra (II)
Architecture

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Data Centers and Racks

- Cassandra is frequently used in systems spanning physically separate locations

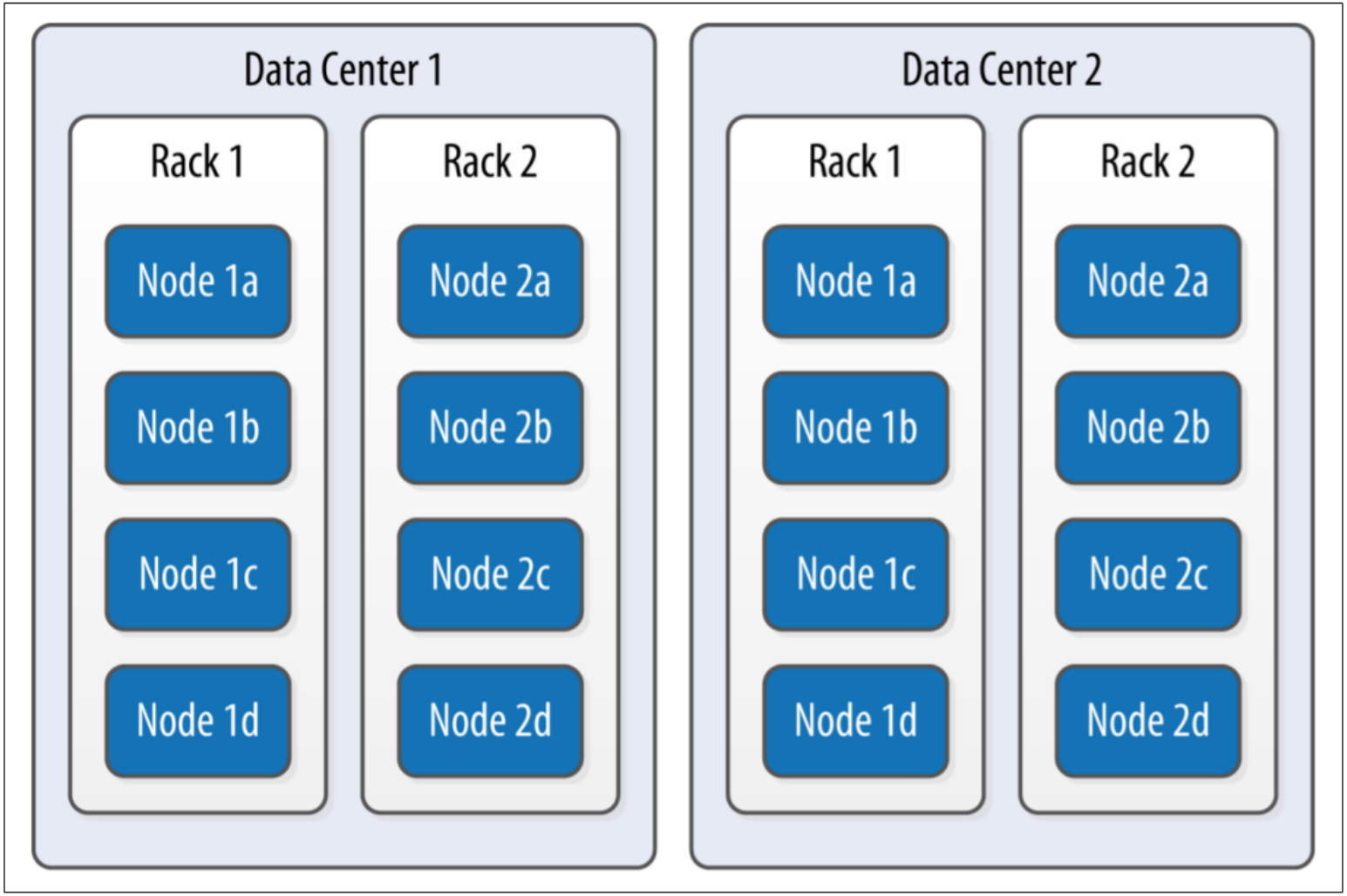
- Cassandra provides two levels of grouping that are used to describe the topology of a cluster:
  - Data center
  - Rack



Data Centers and Racks

- A rack
  - Is a logical set of nodes in close proximity to each other
  - Is located on physical machines in a single rack of equipment
- A data center
  - Is a logical set of racks
  - Is located
    - In the same building
    - At different locations
  - The racks are connected by reliable network

Data Centers and Racks





Data Centers and Racks

- Cassandra leverages the information you provide about your cluster's topology to determine
  - Where to store data
  - How to route queries efficiently
- Cassandra tries to store copies of your data in multiple data centers
  - To maximize availability
  - For partition tolerance
- Cassandra prefers to route queries to nodes in the local data center to maximize performance



Data Centers and Racks

- There are several key use cases for deploying Cassandra across multiple data centers:
  - Live backup
  - Failover
  - Load balancing
  - Geographic distribution
  - Online analysis

Data Centers and Racks
- Live backups -

- Traditional database backups involve:
  - Taking periodic snapshots of the data
  - Storing them offsite in case the system fails
- In the case of failure there will be downtime as a new system is brought up and the data is restored
- This strategy also inevitably leads to data loss for the time period between the last backup and the point of failure
- Snapshot backups are still useful to protect against:
  - Data corruption
  - Accidental updates



Data Centers and Racks
- Live backups -

- Cassandra's data center support can be used to provide a current backup for cases such as hardware failures
- The basic idea involves setting up a second data center that maintains a current set of replicas
- These replicas can be used to rebuild the primary cluster, should a catastrophic event cause the loss of an entire data center
- It is typically enough to maintain a smaller cluster with a replication factor of 1
- This system will never be used to accept live reads or writes
- The primary cost in this case is storage capacity (the replicas needs handle the same quantity of data as the live data center)



Data Centers and Racks
- Failover -

- The failover scenario is very similar to the backup use case

- In this case, the backup data center is generally allocated similar resources as the primary cluster:
  - For a simple backup center: a single replica may be enough
  - For a failover data center: all nodes should be configured with the same replication factor as the primary

- With the failover data center strategy, any node can take over responsibility for the full application workload in the event of a failure



Data Centers and Racks - Failover -

- When you build a failover data center:
  - Consider whether you expect your failover data center to handle the full production workload
  - In this case, you need to ensure that the cluster has adequate capacity to handle the entire production workload
- Having a hot failover data center protects the data center from a common single point of failure (e.g., the power supply to the hosts)



Data Centers and Racks
- Failover -

- Failover considerations:
  - It would be ill-advised to assume you can maintain a small failover data center, and then simply add multiple nodes in the case of failure
  - The additional overhead of bootstrapping the new nodes would reduce the performance at a critical point when the performance is needed most

- Bootstrapping refers to the process of loading the basic software into the memory of a computer after power-on or general reset, especially the operating system which will then take care of loading other software as needed.



Data Centers and Racks
- Load balancing -

- Applications may be configured to route traffic to any node in the cluster without considering a specific data center
- This has the effect of load balancing the requests across multiple data centers
- Can be useful in cases where the data centers share a high-bandwidth connection
- The objective is to provide redundancy:
  - Each data center must be able to handle the entire application workload
  - Similar to the failover scenario



Data Centers and Racks - Load balancing -

- Load balancing considerations:
  - Absolute consistency is expensive to guarantee for load balancing:
    - Consistency requires replicating the data across the entire cluster
    - For strong consistency, employing a geographic distribution model is a good strategy
  - Load balancing is appropriate for use cases where eventual consistency is acceptable:
    - The primary read case involves offline data analysis rather than real-time queries
    - Application examples: event capture, time-series data, logging, etc.



Data Centers and Racks - Geographic distribution -

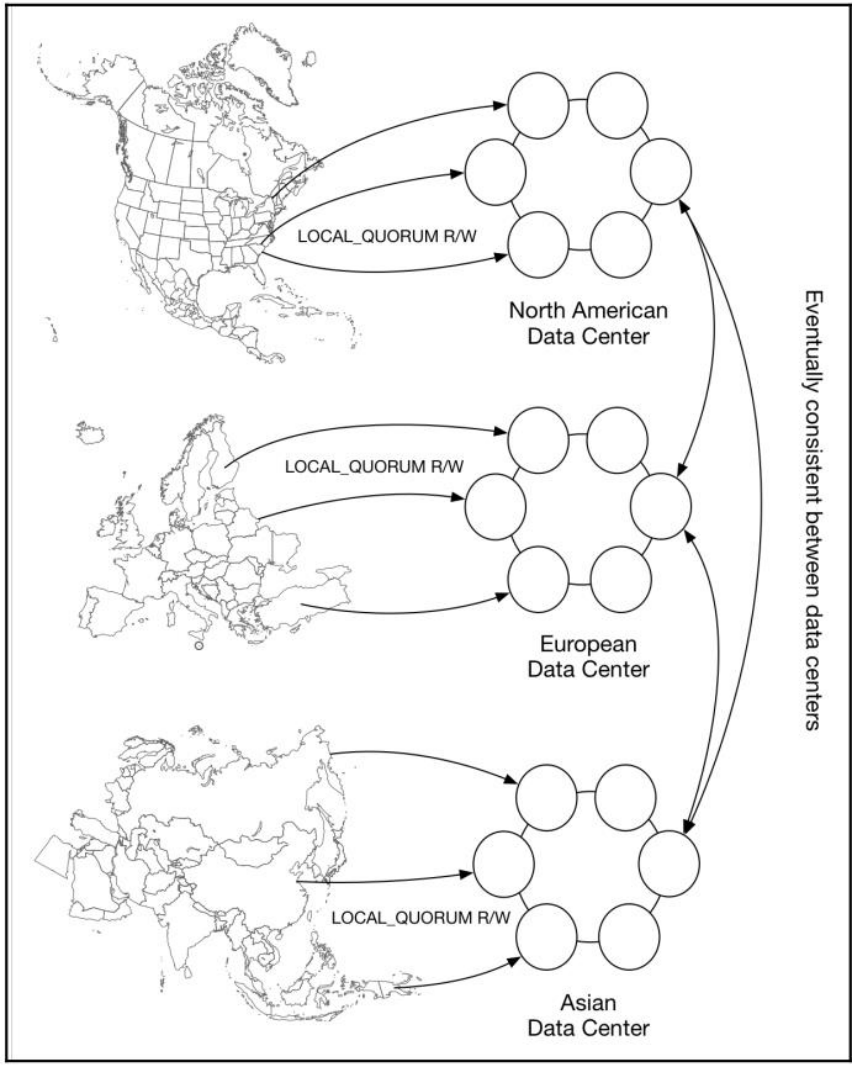
- Some application find it necessary to send requests to a data center located near the application client
- Geographic distribution is useful for systems that span the globe
- Routing all requests to a central location is impractical
- Reasons for geographic distribution
  - To reduce the latency between the originator (application clients) and data centers
  - To mitigate the potential impact of natural disasters

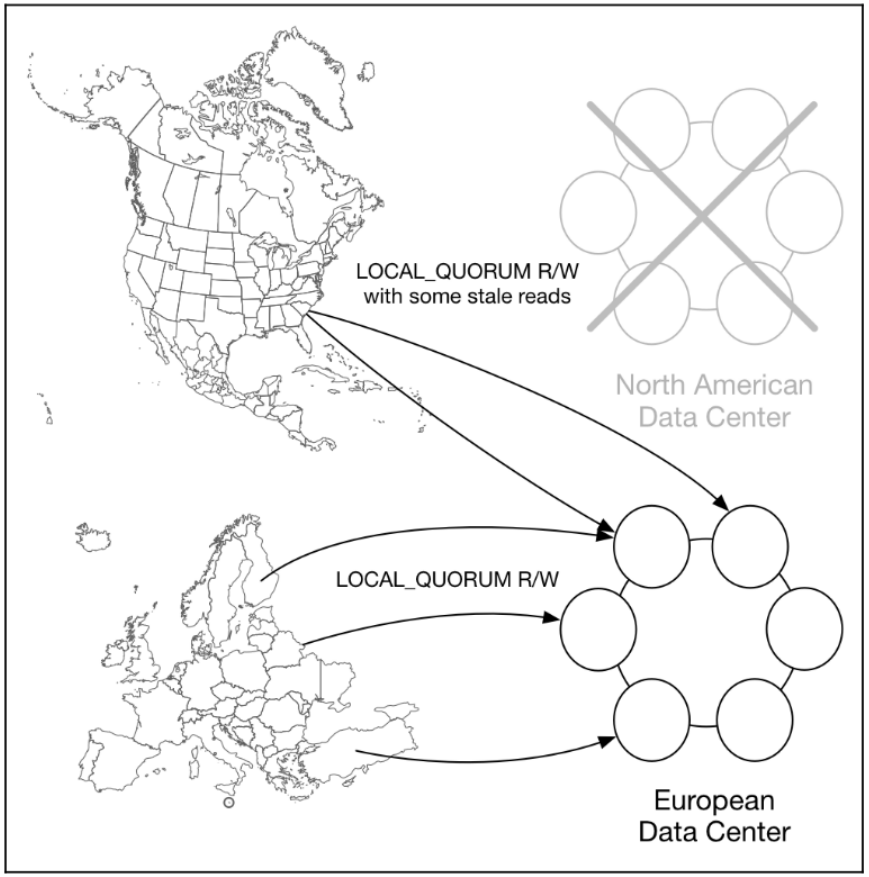


Data Centers and Racks - Geographic distribution -

- Features of geographic distribution
  - The ability to locate data centers in strategic global locations around the world
  - Improve performance
  - Support strong consistency
- To take advantage of these features:
  - Enable the use of the LOCAL_QUORUM consistency level to remove the performance penalty introduced by waiting for a remote data center to acknowledge the write

Data Centers and Racks
- Geographic distribution -





Data Centers and Racks
- Data analysis -

- Data analysis (Online Analytical Processing – OLAP) involves:
  - Taking normalized data from the transactional relational database
  - Moving it into a denormalized form for faster analysis
- This process involves significant Extract Transform and Load (ETL) overhead
- ETL inherently results in a delay in analyzing the data



Data Centers and Racks
- Data analysis -

- Cassandra offers for data analysis:
  - Support of multiple data centers
  - Support replication for load balancing
  - Robust integrations with the Hadoop
  - Robust integrations with the Spark



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Gossip and Failure Detection

- Cassandra uses a gossip protocol to support decentralization and partition tolerance
- The gossip protocol allows each node to keep track of state information about the other nodes in the cluster
- The gossiper runs every second on a timer
- The gossip protocols take their name from the concept of human gossip
  - A form of communication in which peers can choose with whom they want to exchange information



Gossip and Failure Detection

- The gossip protocol:
  - Generally assumes a faulty network
  - Is commonly employed in very large, decentralized network systems
  - Is often used as an automatic mechanism for replication in distributed databases
  - In Cassandra is primarily implemented by the org.apache.cassandra.gms.Gossiper class, which is responsible for managing gossip for the local node



Gossip and Failure Detection

- Cassandra gossip is used for failure detection
- The Gossiper class maintains a list of nodes that are alive and dead
- When a server node is started:
  - It registers itself with the gossiper
  - It will receive from the gossiper the endpoint state information



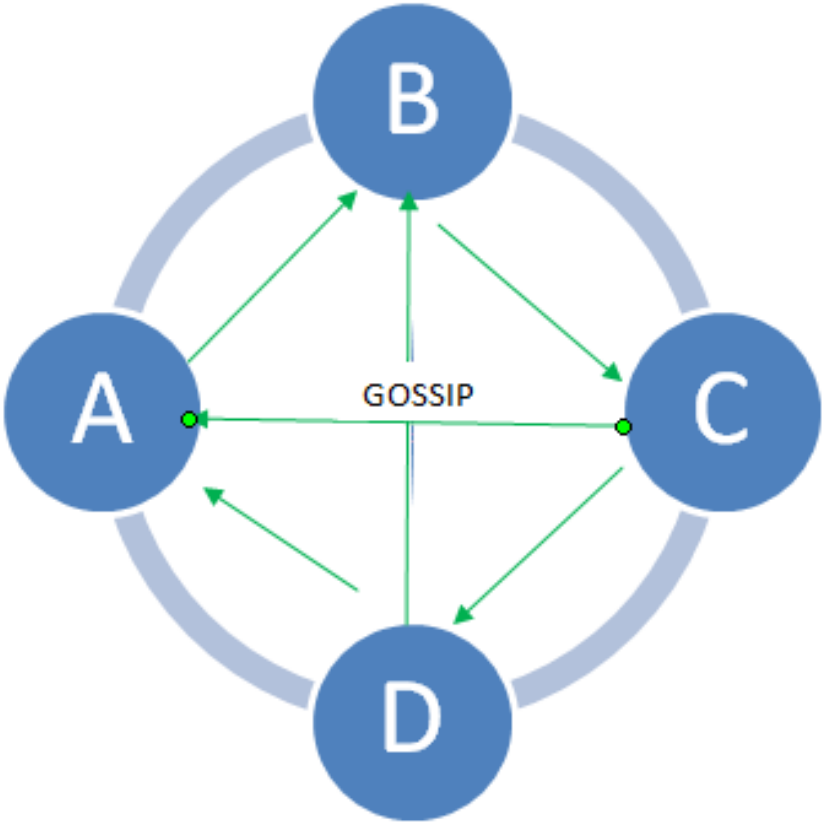
Gossip and Failure Detection

- The gossiper works as follows:

1. Once per second, the gossiper:
   I. Will choose a random node in the cluster (friend)
   II. Will initialize a gossip session with the chosen friend

2. Each round of gossip requires three messages:
   I. The gossip initiator sends its chosen friend a GossipDigestSynMessage
   II. When the friend receives this message, it returns a GossipDigestAckMessage
   III. When the initiator receives the ack message from the friend, it sends the friend a GossipDigestAck2Message to complete the round of gossip.

Gossip and Failure Detection



PEER TO PEER DISTRIBUTION
MODEL OF CASSANDRA



Gossip and Failure Detection

- When the gossiper determines that another endpoint is dead:
  - "Convicts" that endpoint
  - Marks it as dead in its local list
  - Logs the modification



Gossip and Failure Detection

- Cassandra has robust support for failure detection
- Failure detections uses the algorithm for distributed computing called Phi Accrual Failure Detection.



Gossip and Failure Detection

- Phi Accrual Failure Detectors
  - Accrual Failure Detectors (AFD) output a value associated with each process (or node)
  - This value is called Phi
  - The value is output in a manner that is designed from the ground up to be adaptive in the face of volatile network conditions
  - Phi is not a binary condition that simply checks whether a server is up or down
  - The Phi convict threshold in the configuration adjusts the sensitivity of the failure detector.



Gossip and Failure Detection

- Phi Accrual Failure Detectors
  - Lower values increase the sensitivity to failure
  - Higher values decrease the sensitivity to failure
  - This process is not done in a linear fashion
  - The Phi value refers to a level of suspicion that a server might be down
  - Applications such as Cassandra that employ an AFD can specify variable conditions for the Phi value they emit
  - Cassandra can generally detect a failed node in about 10 seconds using this mechanism



Gossip and Failure Detection

- Cassandra's accrual failure detection is based on two primary ideas:

1. The first general idea is that failure detection should be flexible
   - This is achieved by decoupling it from the application being monitored

2. The second and more novel idea is using a suspicion level instead of a binary approach (i.e., dead or alive)
   - Challenges the notion of traditional failure detectors
   - Traditional failure detectors
     - Are implemented by simple "heartbeats"
     - Decide whether a node is dead or not dead based on whether a heartbeat is received or not
   - AFD decides that this approach is naive and finds a place in between the extremes of dead and alive - a suspicion level



Gossip and Failure Detection

- Cassandra's accrual failure
  - The failure monitoring system outputs a continuous level of "suspicion" regarding how confident it is that a node has failed.
  - This is desirable because it can consider fluctuations in the network environment
  - For example, just because one connection gets caught up doesn't necessarily mean that the whole node is dead
  - So suspicion offers a more fluid and proactive indication of the weaker or stronger possibility of failure based on interpretation (the sampling of heartbeats), as opposed to a simple binary assessment



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Rings and Tokens

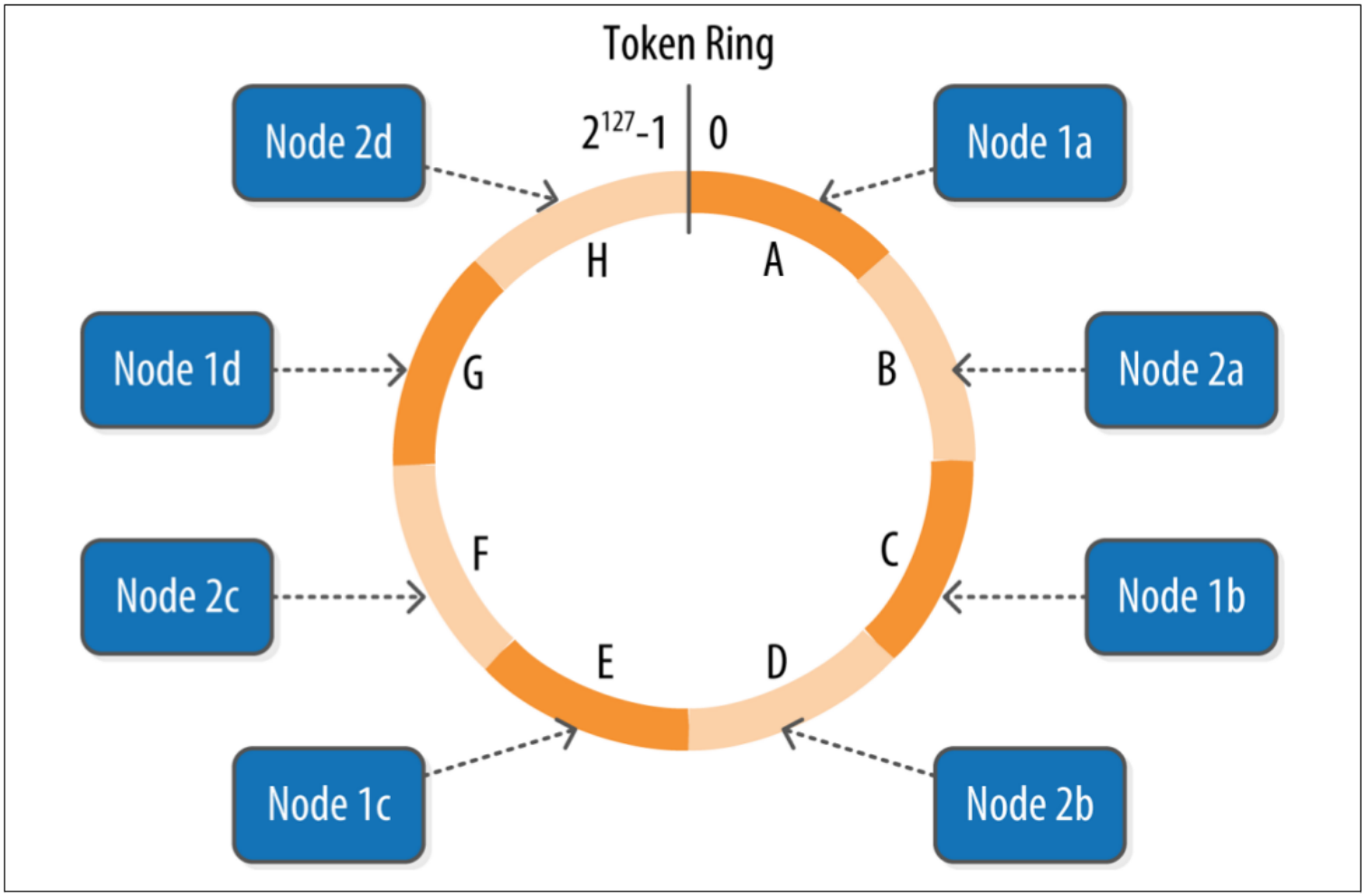
- Cassandra represents the data managed by a cluster as a ring
- Each node in the ring is assigned one or more ranges of data described by a token
- The token determines the nodes position in the ring
- A token is a 64-bit integer ID used to identify each partition
- This gives a possible range for tokens in the range $[-2^{64}, 2^{63}-1]$



Rings and Tokens

- A node claims ownership of the range of values
  - Less than or equal to each token, and
  - Greater than the token of the previous node
- The node with lowest token owns the range
  - Less than or equal to its token, or
  - Greater than the highest token
  - This range is also known as the "wrapping range"
- In this way, the tokens specify a complete ring

Rings and Tokens





Rings and Tokens

- The ring is structured such that consecutive token ranges are spread across nodes in different racks
- Data is assigned to nodes by using a hash function to calculate a token for the partition key
- This partition key token is compared to the token values for the various nodes to identify the range, and therefore the node, that owns the data



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Virtual Nodes

- Cassandra's 1.2 release introduced the concept of virtual nodes, also called vnodes for short
- Instead of assigning a single token to a node, the token range is broken up into multiple smaller ranges
- Each physical node is then assigned multiple tokens
- By default, each node will be assigned 256 of these tokens, meaning that it contains 256 virtual nodes



Virtual Nodes

- Virtual nodes have been enabled by default since 2.0.
- Vnodes make it easier to maintain a cluster containing heterogeneous machines
- For nodes in your cluster that have more computing resources available to them, you can increase the number of vnodes by setting the num_tokens property in the cassandra.yaml file
- Conversely, you might set num_tokens lower to decrease the number of vnodes for less capable machines



Virtual Nodes

- Cassandra automatically handles the calculation of token ranges for each node in the cluster in proportion to their num_tokens value

- A further advantage of virtual nodes is that they speed up some of the more heavy-weight Cassandra operations such as
  - Bootstrapping a new node
  - Decommissioning a node
  - Repairing a node

- This improvement is because the workload associated with operations on multiple smaller ranges is spread more evenly across the nodes in the cluster



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Partitioners

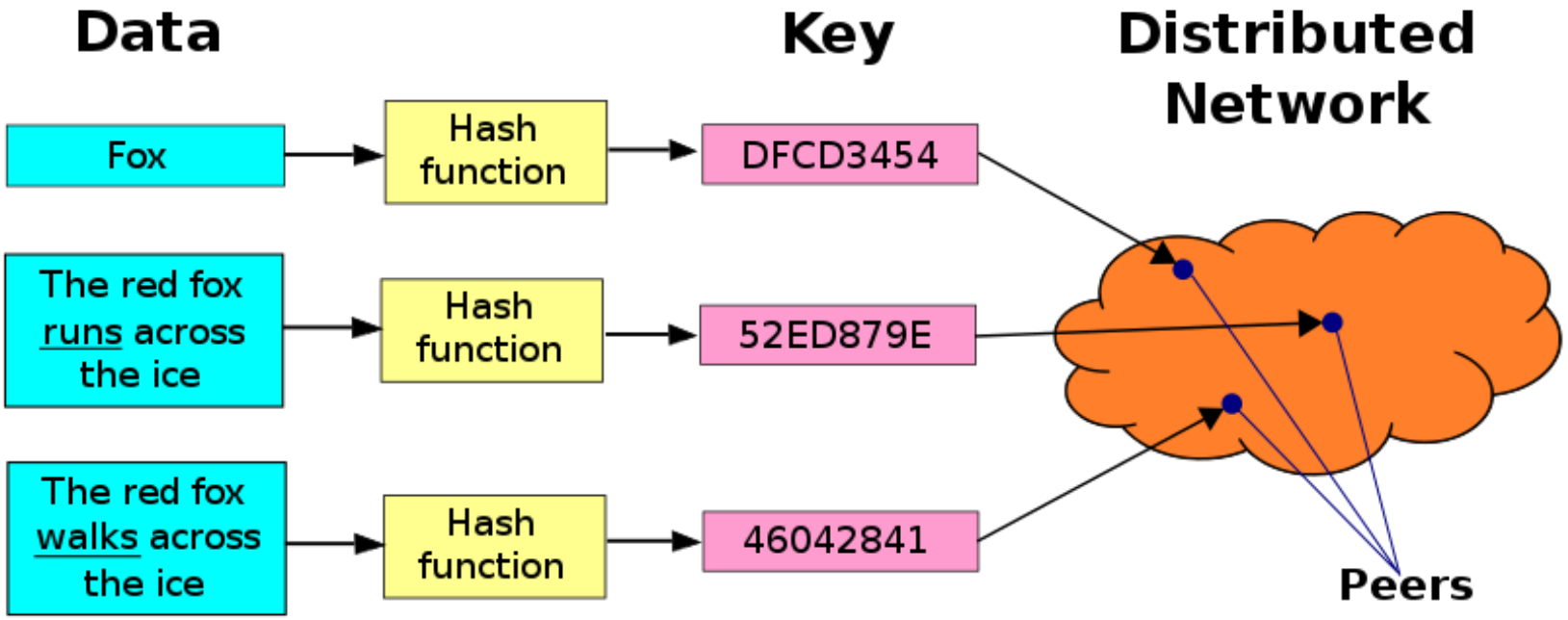
- A partitioner determines how data is distributed across the nodes in the cluster
- Cassandra stores data in wide rows, or "partitions"
- Each row has a partition key that is used to identify the partition
- A partitioner is a hash function for computing the token of a partition key
- Each row of data is distributed within the ring according to the value of the partition key token



Partitioners

- Cassandra provides several different partitioners that use a distributed hash table (DHT)
- A DHT is a class of a decentralized distributed system that provides a lookup service similar to a hash table
- For hashing, Cassandra uses a Java implementation of the murmur algorithm that generates 64-bit hashes
- MurmurHash is a non-cryptographic hash function suitable for general hash-based lookup
- Because of Cassandra's generally pluggable design, you can also create your own partitioner

Partitioners





Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Replication Strategies

- A node serves as a replica for different ranges of data
- If one node goes down, other replicas can respond to queries for that range of data
- Cassandra replicates data across nodes in a manner transparent to the user using a replication factor
- The replication factor is the number of nodes in your cluster that will receive copies (replicas) of the same data



Replication Strategies

- If the replication factor is 3, then three nodes in the ring will have copies of each row
- The first replica will always be the node that claims the range in which the token falls
- The remainder of the replicas are placed according to the replication strategy (or replica placement strategy)



Replication Strategies

- Cassandra provides two primary replication strategies:

1. The SimpleStrategy
   - Places replicas at consecutive nodes around the ring
   - It starts with the node indicated by the partitioner

2. The NetworkTopologyStrategy
   - That allows the administrator to specify a different replication factor for each data center
   - Within a data center, it allocates replicas to different racks in order to maximize availability



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Consistency Levels

- Cassandra provides tuneable consistency levels that allow you to make trade-offs between consistency, availability, and partition tolerance at a fine-grained level
- Cassandra lets the client specify a consistency level on each read or write query that indicates how much consistency you require
- A higher consistency level means that more nodes need to respond to a read or write query, ensuring that the values present on each replica are the same



Consistency Levels

- For read queries: the consistency level specifies how many replica nodes must respond to a read request before returning the data
- For write operations: the consistency level specifies how many replica nodes must respond for the write to be reported as successful to the client
- Because Cassandra is eventually consistent: updates to other replica nodes may continue in the background



Consistency Levels

- The available consistency levels include
  - ANY, ONE, TWO, and THREE: specify an absolute number of replica nodes that must respond to a request
  - QUORUM consistency level requires a response from a majority of the replica nodes (N is the replication factor): N/2 + 1
  - ALL consistency level requires a response from all the replicas

- For both reads and writes, the consistency levels of
  - ANY, ONE, TWO , and THREE are considered weak
  - QUORUM and ALL are considered strong



Consistency Levels

- Consistency is tuneable in Cassandra because clients can specify the desired consistency level on both reads and writes
- There is an equation that is popularly used to represent the way to achieve strong consistency in Cassandra: R + W > N = strong consistency
- In this equation
  - R the read replica count
  - W the write replica count
  - N the replication factor
- All client reads will see the most recent write in this scenario, and you will have strong consistency



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Queries and Coordinator Nodes

- A client may connect to any node in the cluster to initiate a read or write query
- This node is known as the coordinator node
- The coordinator
  - Identifies which nodes are replicas for the data that is being written or read
  - Forwards the queries to the replicas



Queries and Coordinator Nodes

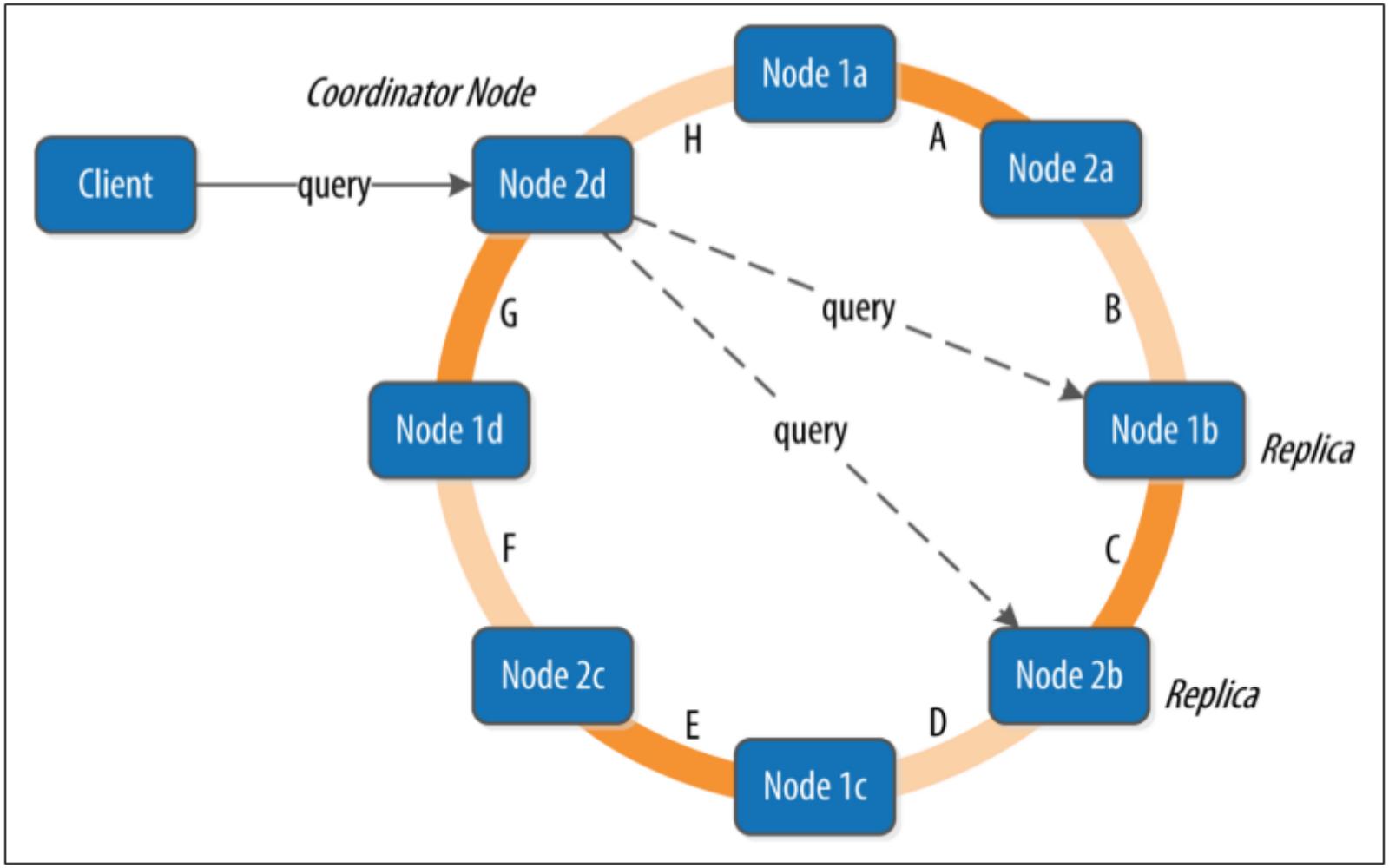
- Write:

1. The coordinator node contacts all replicas (as determined by the consistency level and replication factor)
2. Considers the write successful when a number of replicas commensurate with the consistency level acknowledge the write

- Read:

1. The coordinator contacts enough replicas to ensure the required consistency level is met
2. Returns the data to the client

Queries and Coordinator Nodes





Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff

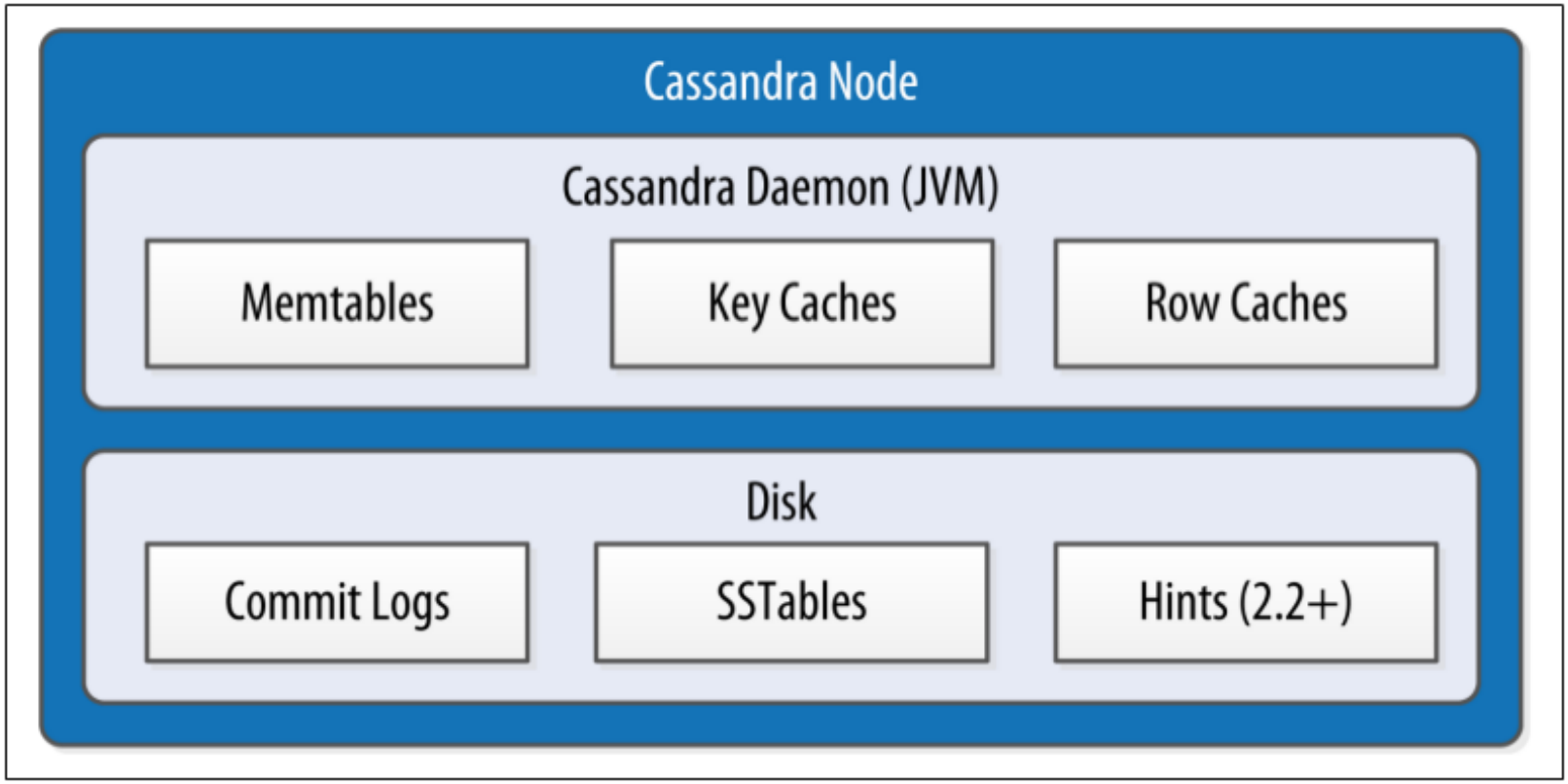


Commit Logs, Memtables and SSTables

- Cassandra stores data both in memory and on disk to provide both high performance and durability

- To support the writing and reading of data from tables, Cassandra uses
  1. Commit logs
  2. Memtables
  3. SSTables

Commit Logs, Memtables and SSTables





Commit Logs

- The commit log is a crash-recovery mechanism that supports Cassandra's durability goals
- When a write operation is performed, it is immediately written to a commit log
- A write will not count as successful until it's written to the commit log
- This ensures that if a write operation does not make it to the in-memory store (the memtable), it will still be possible to recover the data
- The commit log can ensure that data is not lost if the database is shut down or it crashes unexpectedly
  - That's because the next time you start the node, the commit log gets replayed
  - That's the only time the commit log is read – clients never read from it



Commit Logs

- Each commit log maintains an internal bit flag to indicate whether it needs flushing
- When a write operation is first received, it is written to the commit log and its bit flag is set to 1
- There is only one bit flag per table, because only one commit log is ever being written to across the entire server
- All writes to all tables will go into the same commit log, so the bit flag indicates whether a particular commit log contains anything that hasn't been flushed for a particular table
- After it's written to the commit log, the value is written to a memory-resident data structure called the memtable



Commit Logs

- Once the memtable has been properly flushed to disk
  - The corresponding commit log's bit flag is set to 0
  - Indicating that the commit log no longer must maintain that data for durability purposes

- Like regular logfiles
  - Commit logs have a configurable rollover threshold
  - Once this file size threshold is reached:
    - The log will roll over
    - It carrys with it any extant dirty bit flags



Memtables

- Cassandra stores data in-memory using memtables
- Each memtable contains data for a specific table
- In early implementations of Cassandra, memtables were stored on the JVM heap, but now the majority of memtable data was moved to native memory
- This makes Cassandra less susceptible to fluctuations in performance due to Java garbage collection



Memtables

- When the number of objects stored in the memtable reaches a threshold
  - The contents of the memtable are flushed to disk
  - On the disk, the data are stored in a file called an SSTable
  - A new memtable is then created
- The flushing operation is a non-blocking:
  - Multiple memtables may exist for a single table (i.e., one current and the rest waiting to be flushed)
  - The memtables typically should not have to wait very long for flushing: the node should flush them very quickly unless it is overloaded



SSTables

- Once a memtable is flushed to disk as an SSTable
  - It is immutable
  - Cannot be changed by the application
- SSTables are compacted:
  - This compaction changes only their on-disk representation
  - It essentially performs the "merge" step of a mergesort into new files and removes the old files on success
- Cassandra supports the compression of SSTables
  - To maximize use of the available storage
  - This compression is configurable per table
- Each SSTable also has an associated Bloom filter used as an additional performance enhancer (a Bloom filter is a data structure designed to tell you, rapidly and memory-efficiently, whether an element is present in a set)



SSTables

- All writes are sequential, which is the primary reason that writes perform so well in Cassandra
- No reads or seeks of any kind are required for writing a value to Cassandra because all writes are append operations
- One key limitation on performance is the speed of the hard disk
- Compaction is intended to amortize the reorganization of data, but it uses sequential I/O to do so



SSTables

- Write operations
  - Write operations are just an immediate append
  - Compaction helps to organize for better future read performance
  - If Cassandra naively inserted values where they ultimately belonged: writing clients would pay for seeks up front

- Read operations
  - Cassandra will read both SSTables and memtables to find data values
  - Memtable may contain values that have not yet been flushed to disk



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Caching

- Cassandra provides three forms of caching:

1. Key caches
   - Stores a map of partition keys to row index entries
   - Facilitating faster read access into SSTables stored on disk
   - The key cache is stored on the JVM heap

2. Row caches
   - Uses entire rows
   - Can greatly speed up read access for frequently accessed rows, at the cost of more memory usage
   - The row cache is stored in off-heap memory

3. The counter cache
   - Developed to improve counter performance by reducing lock contention for the most frequently accessed counters
   - Lock contention takes place when a thread tries to acquire the lock to an object which is already acquired by other thread
   - Counters are used for concurrent increment/decrement on values without requiring read-before-write or locking

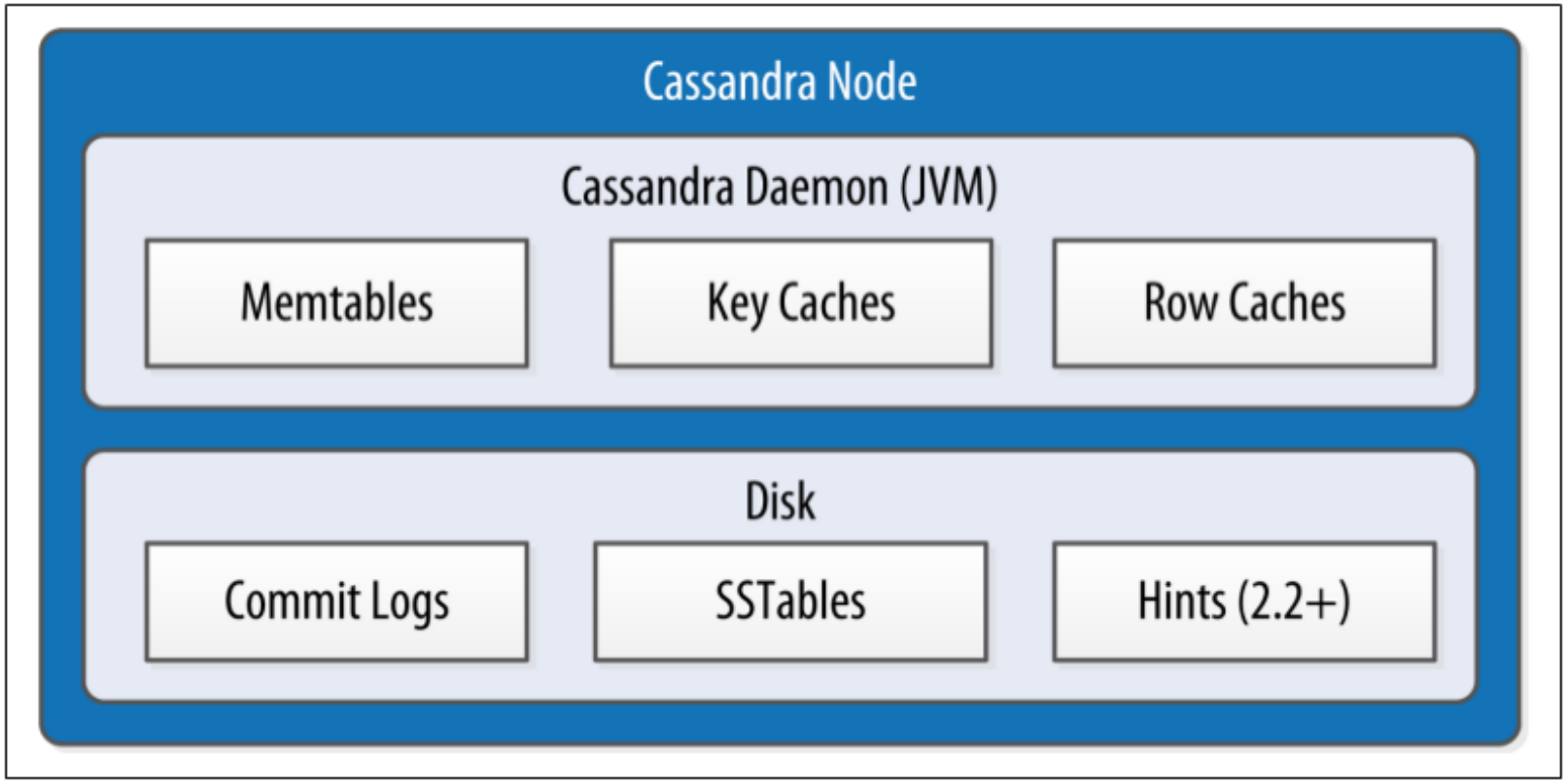


Caching

- By default, key and counter caching are enabled
- Row caching is disabled by default because it requires more memory
- Cassandra saves its caches to disk periodically in order to warm them up more quickly on a node restart



Caching


Cassandra Node
Cassandra Daemon (JVM)
Memtables
Key Caches
Row Caches
Disk
Commit Logs
SSTables
Hints (2.2+)



Overview

- Data Centers and Racks
- Gossip and Failure Detection
- Rings and Tokens
- Vnodes (virtual nodes)
- Partitioners
- Replication Strategies
- Consistency Levels
- Queries and Coordinator Nodes
- Commit Logs, Memtables, and SSTables
- Caching
- Hinted Handoff



Hinted Handoff

- If a write request is sent to Cassandra, but a replica node where the write properly belongs is not available (due to network partition, hardware failure, or some other reason) than the coordinator will create a hint

- Hits are used by Cassandra to ensure general availability in such cases

- Cassandra holds a separate hint for each partition that is to be written
  - Allows Cassandra to be always available for writes
  - Enables a cluster to sustain the same write load even when some of the nodes are down
  - Reduces the time that a failed node will be inconsistent after it does come back online



Hinted Handoff

